联邦学习基本架构

一、损失函数 (Loss Function) 的核心概念

在机器学习中，**损失函数**（有时也称为代价函数，Cost Function）是一个衡量模型预测值与真实值之间差异的数学函数。它的核心作用是为模型的学习过程提供一个**明确的优化目标**：损失函数的值越小，说明模型的预测越准确，性能越好。

数学表达：

假设模型当前的参数为 w，输入数据为 x，真实的标签为 y，模型给出的预测值为 $\hat{y} = f(w; x)$。

对于单个数据样本，损失函数记为 $l(w; x, y)$，它计算了 $\hat{y}$ 和 y 之间的误差。

在包含 n 个样本的整个数据集上，模型的目标是最小化**经验风险（Empirical Risk）**，即所有样本损失的平均值。这正是前文联邦学习推导中提到的目标函数 $F(w)$：

$$F(w) = \frac{1}{n} \sum_{i=1}^{n} l(w; x_i, y_i)$$

常见分类：

- **回归问题**：常使用均方误差（Mean Squared Error, MSE），$l = (\hat{y} - y)^2$。
- **分类问题**：常使用交叉熵损失（Cross-Entropy Loss），$l = -\sum y \log(\hat{y})$。

二、损失函数与梯度的关系

如果把损失函数 $F(w)$ 想象成一片连绵起伏的山地，山地上的高度代表损失值的大小，那么模型的训练过程就是“寻找山谷最低点（损失最小值）”的过程。在这个过程中，**梯度（Gradient）** 充当了“指南针”的角色。

梯度的数学定义：

梯度是损失函数对模型所有参数求偏导数构成的向量。假设模型参数 w 是一个包含 d 个维度的向量 $w = [w_1, w_2, \ldots, w_d]^T$，那么损失函数 $F(w)$ 在当前参数点处的梯度表示为 $\nabla F(w)$：

$$\nabla F(w) = \left[\frac{\partial F}{\partial w_1}, \frac{\partial F}{\partial w_2}, \ldots, \frac{\partial F}{\partial w_d} \right]^T$$

两者的核心联系：

1. **梯度的几何意义**：在损失函数构成的多维空间中，梯度向量的方向始终指向当前位置损失值**上升最快**的方向。梯度的模长（大小）表示上升的陡峭程度。
2. **梯度下降法（Gradient Descent）**：既然优化的目标是让损失函数变小，我们只需要让参数沿着梯度的**反方向**（即下降最快的方向）移动一小步。这就是模型参数更新的核心公式：

3. $w_{new} = w_{old} - \eta \nabla F(w_{old})$
4. 其中，η 是学习率（步长），负号代表逆着梯度方向移动。当梯度接近于 0 （零向量）时，意味着到达了平坦的谷底（局部极小值或全局最小值），此时参数不再发生明显更新，模型完成收敛。

三、在联邦学习框架下的意义

将损失函数与梯度的关系代入到前文的联邦学习推导中，可以更清晰地理解为何要进行“参数聚合”或“梯度聚合”：

在联邦学习中，全局目标是最小化所有客户端数据的总损失函数 $F(w)$。然而，由于数据隐私限制，中央服务器无法直接计算全局梯度 $\nabla F(w)$。

由于导数具有**线性相加**的性质，全局损失的梯度等于各个客户端本地损失梯度的加权和：

$$\nabla F(w) = \nabla \left(\sum_{k=1}^{K} p_k F_k(w) \right) = \sum_{k=1}^{K} p_k \nabla F_k(w)$$

这就是 **FedSGD** 中服务器直接对客户端上传的本地梯度 $g_k = \nabla F_k(w)$ 求加权平均的数学基础。而在 **FedAvg** 中，客户端在本地目标函数 $F_k(w)$ 的山谷里用局部梯度多走了几步，虽然这节省了通信，但由于各个客户端的山谷形状不同（Non-IID 数据造成的损失函数面貌差异），多走几步就容易偏离全局山谷的中心，这就是产生**客户端漂移**的根本几何原因。

公式符号系统

为保证推导的严谨性与一致性，统一使用以下数学符号描述联邦学习系统：

- K：参与联邦学习的客户端（Client）总数。
- k：客户端索引，$k \in \{1, 2, \dots, K\}$。
- n_k：第 k 个客户端本地的数据样本量。
- n：所有参与客户端的总样本量，即 $n = \sum_{k=1}^{K} n_k$。
- p_k：第 k 个客户端在全局聚合时的权重参数，满足 $p_k = \frac{n_k}{n}$ 且 $\sum_{k=1}^{K} p_k = 1$。
- w：机器学习模型的参数（权重向量）。
- w^t：第 t 轮通信开始时的全局模型参数。
- w_k^{t+1}：第 t 轮通信中，第 k 个客户端经过本地训练后准备上传的本地模型参数。
- $F_k(w)$：第 k 个客户端的本地损失函数。全局损失函数为 $F(w) = \sum_{k=1}^{K} p_k F_k(w)$。
- η：本地训练的学习率。

- g_k：本地损失函数对模型参数的梯度，即 $\nabla F_k(w)$。
- τ_k：第 k 个客户端在单轮通信中执行的本地梯度下降步数。

总体更新公式

FedSGD $w^{t+1} = w^t - \eta \sum_{k=1}^{K} p_k g_k^t$

FedAvg $w^{t+1} = w^t - \eta \sum_{k=1}^{K} p_k \left(\sum_{i=0}^{\tau-1} g_{k,i} \right)$

1. FedSGD (Federated Stochastic Gradient Descent)

FedSGD 是最基础的分布式优化算法，客户端在本地只执行**单步**梯度下降。

本地更新过程：

第 t 轮中，客户端 k 接收到全局模型 w^t，在本地数据上计算梯度 $g_k^t = \nabla F_k(w^t)$，并执行一步参数更新：

$$w_k^{t+1} = w^t - \eta g_k^t$$

全局聚合推导：

客户端将更新后的权重 w_k^{t+1} 上传给服务器。服务器执行加权平均：

$$w^{t+1} = \sum_{k=1}^{K} p_k w_k^{t+1}$$

将本地更新公式代入聚合公式进行完整展开：

$$
\begin{aligned}
w^{t+1} &= \sum_{k=1}^{K} p_k \left(w^t - \eta g_k^t \right) \\
&= \sum_{k=1}^{K} \left(p_k w^t - p_k \eta g_k^t \right) \\
&= \sum_{k=1}^{K} p_k w^t - \sum_{k=1}^{K} p_k \eta g_k^t \\
&= w^t \sum_{k=1}^{K} p_k - \eta \sum_{k=1}^{K} p_k g_k^t
\end{aligned}
$$

由于 $\sum_{k=1}^{K} p_k = 1$，最终化简为：

$$
w^{t+1} = w^t - \eta \sum_{k=1}^{K} p_k g_k^t
$$

推导结论： 对客户端上传的**模型权重**求加权平均，在数学上完全等价于对客户端的**本地梯度**求加权平均并在服务器端执行一次联合梯度下降。

1. 明文场景 (Plaintext)

- **形式 A（传权重）：**
  - 本地计算：$w_k^{t+1} = w^t - \eta g_k^t$
  - 上传内容：单步更新后的模型权重 w_k^{t+1}
  - 全局聚合：$w^{t+1} = \sum_{k=1}^{K} p_k w_k^{t+1}$
- **形式 B（传梯度 —— 理论文献最常见）：**
  - 本地计算：单步梯度 g_k^t。为了分担服务器计算，客户端可提前算出**加权梯度** $p_k g_k^t$。
  - 上传内容：g_k^t 或 $p_k g_k^t$
  - 全局聚合：$w^{t+1} = w^t - \eta \sum_{k=1}^{K} p_k g_k^t$

2.密文场景 (Ciphertext / CKKS)

- **本地预加权与加密**：客户端直接在本地执行纯明文乘法，算出加权梯度 $p_k g_k^t$。随后打包并加密，上传密文 $[\![p_k g_k^t]\!]$。
- **Zero-Depth 聚合**：服务器收到后，由于权重已在明文态消化，仅需执行同态加法：

- $[\![\Delta_{global}]\!] = \sum_{k=1}^{K} [\![p_k g_k^t]\!]$
- 随后执行全局更新：$[\![w^{t+1}]\!] = [\![w^t]\!] - \eta [\![\Delta_{global}]\!]$。

2. FedAvg (Federated Averaging)

为了大幅降低通信开销，FedAvg 允许客户端在本地执行 τ 步连续的梯度下降后再进行通信。

本地多步更新展开：

客户端 k 接收全局模型，初始化本地参数 $w_{k,0} = w^t$。随后执行 τ 步更新：

- 第 1 步：$w_{k,1} = w_{k,0} - \eta g_{k,0} = w^t - \eta g_{k,0}$
- 第 2 步：$w_{k,2} = w_{k,1} - \eta g_{k,1} = w^t - \eta(g_{k,0} + g_{k,1})$
- 第 3 步：$w_{k,3} = w^t - \eta(g_{k,0} + g_{k,1} + g_{k,2})$

  ……
- 以此类推，第 τ 步为：

$$w_k^{t+1} = w_{k,\tau} = w^t - \eta \sum_{i=0}^{\tau-1} g_{k,i}$$

全局聚合推导：

服务器对所有客户端上传的最终模型 w_k^{t+1} 执行加权平均。我们将上述多步更新的结果代入：

$$\begin{aligned} w^{t+1} &= \sum_{k=1}^{K} p_k w_k^{t+1} \\ &= \sum_{k=1}^{K} p_k \left(w^t - \eta \sum_{i=0}^{\tau-1} g_{k,i} \right) \\ &= \sum_{k=1}^{K} p_k w^t - \sum_{k=1}^{K} p_k \eta \sum_{i=0}^{\tau-1} g_{k,i} \\ &= w^t - \eta \sum_{k=1}^{K} p_k \left(\sum_{i=0}^{\tau-1} g_{k,i} \right) \end{aligned}$$

推导结论： FedAvg 的全局更新本质上是沿着所有客户端**本地累积梯度**的加权平均方向进行下降。如果定义模型增量 $\Delta_k = w_k^{t+1} - w^t = -\eta \sum_{i=0}^{\tau-1} g_{k,i}$，聚合公式可以简洁地写为 $w^{t+1} = w^t + \sum_{k=1}^{K} p_k \Delta_k$。

对FedAvg来说，

1、在明文上传的东西也可以有两种形式：

根据之前的展开推导，客户端在本地走完 τ 步后，最终的模型增量 Δ_k 等于这 τ 步梯度的累加再乘以学习率：

$$\Delta_k = w_k^{t+1} - w^t = -\eta \sum_{i=0}^{\tau-1} g_{k,i}$$

- **选择A（传权重）：**

  本地计算：$w_k^{t+1} = w_{k,\tau} = w^t - \eta \sum_{i=0}^{\tau-1} g_{k,i}$

  上传内容：单步更新后的模型权重 w_k^{t+1}

  全局聚合：$w^{t+1} = \sum_{k=1}^{K} p_k w_k^{t+1}$

- **选择 B（传增量）：** 客户端直接上传算好的 $\Delta_k = -\eta \sum_{i=0}^{\tau-1} g_{k,i}$。然后上传 Δ_k。

  服务器直接相加：$w^{t+1} = w^t + \sum p_k \Delta_k$。

- **选择 C（传累积梯度）：** 客户端把 τ 步的梯度累加，记为 $G_k = \sum_{i=0}^{\tau-1} g_{k,i}$，然后上传 G_k

  服务器收到后，在服务端乘以学习率更新：$w^{t+1} = w^t - \eta \sum p_k G_k$。

这两种方式在数学上是百分之百等价的。因此，**传累积梯度完全可行，也是 FedAvg 的合法表达形式。**

2.在密文上上传：

本地明文吸收 η 和 p_k：

客户端不仅不要传累积梯度，甚至不要传纯净的增量。客户端获取到自己的权重 p_k 后，在本地直接计算出**加权后的最终增量**：

$$\tilde{\Delta}_k = p_k \cdot \Delta_k = p_k \cdot \left(-\eta \sum_{i=0}^{\tau-1} g_{k,i} \right)$$

在这里，学习率 η 和权重 p_k 的乘法全部在 CPU 上以极快的明文浮点数形式瞬间完成，毫无密码学开销。

SIMD 槽打包与加密：

客户端将计算好的 $\tilde{\Delta}_k$ 对齐并打包（Pack）进 CKKS 的明文多项式槽（Slots）中，然后执行加密，生成密文 $[\![\tilde{\Delta}_k]\!]$。

服务器端纯加法聚合（Zero Depth）：

服务器收到密文后，由于所有的系数都已经提前乘好了，服务器要做的**仅仅是同态加法（EvalAdd）**：

又因为：$w^{t+1} = w^t + \sum_{k=1}^{K} p_k \Delta_k = w^t + \sum_{k=1}^{K} \tilde{\Delta}_k$

$$[\![\Delta_{global}]\!] = \sum_{k=1}^{K} [\![\tilde{\Delta}_k]\!]$$

同态加法在 CKKS 中几乎是零成本的，且**完全不消耗任何乘法深度（Depth）**，不需要做 `Rescale`。

在密文上处理 $\tilde{\Delta}_k$：

路线一：客户端本地解密并更新（工业界主流，最推荐）

这也是为什么传增量 Δ_k 在工程上极其优雅的原因。因为每个客户端本地本来就存有一份本轮的起点模型 w^t。

1. **下发增量密文**：服务器**不处理** w^t，直接把聚合好的增量密文 $[\![\Delta_{global}]\!]$ 广播下发给所有客户端。
2. **客户端解密**：客户端拿到密文后，使用自己手中的私钥进行解密，得到明文的全局增量 Δ_{global}。
3. **本地明文更新**：客户端在本地内存中，将解密出的增量加到上一轮的模型上：
4. $w^{t+1} = w^t + \Delta_{global}$
5. **优势**：服务器彻底变成了“无情的密文累加器”，它不仅不知道增量是多少，甚至连全局模型 w^t 本身都不需要维护，极大地降低了服务器的存储和计算负担。

路线二：服务器密态直接更新（Server-side Ciphertext Update）

如果系统设定服务器必须掌握并维护整个全局模型的最新密文状态。

1. **服务器维护密文模型**：服务器本地一直保存着上一轮的全局模型密文 $[\![w^t]\!]$。
2. **密态加法**：聚合出 $[\![\Delta_{global}]\!]$ 后，服务器再额外执行一次同态加法：
3. $[\![w^{t+1}]\!] = [\![w^t]\!] \oplus [\![\Delta_{global}]\!]$
4. **下发完整模型密文**：服务器将最新的 $[\![w^{t+1}]\!]$ 广播给客户端。客户端解密后直接得到最新模型用于下一轮训练。
5. **劣势**：虽然同态加法开销小，但每次下发必须传输**整个模型的密文**。如果在某些系统中（如采用了稀疏化技术），增量的体积远小于完整模型，这条路线会造成严重的下行网络带宽浪费。

3. FedProx (Federated Proximal)

在 Non-IID（数据非独立同分布）场景下，FedAvg 的多步本地更新会导致不同客户端的模型朝各自数据分布的局部极小值收敛，产生**客户端漂移**。FedProx 通过引入近端项（Proximal Term）来约束本地更新幅度。

本地目标函数重构：

客户端 k 的优化目标不再仅仅是原始损失 $F_k(w)$，而是加入了一个 L2 正则项，惩罚本地模型 w 偏离当前全局模型 w^t 的距离：

$$h_k(w) = F_k(w) + \frac{\mu}{2}\|w - w^t\|^2$$

其中 μ 是超参数。该项对参数 w 的导数为 $\mu(w - w^t)$。

本地更新过程：

客户端针对新目标函数 $h_k(w)$ 进行 τ 步梯度下降。在第 i 步迭代时，参数更新公式变为：

$$w_{k,i+1} = w_{k,i} - \eta \left(\nabla F_k(w_{k,i}) + \mu(w_{k,i} - w^t)\right)$$

这个额外的拉力项 $\mu(w_{k,i} - w^t)$ 确保了即使进行了多步更新，本地模型也不会偏离全局模型太远。

全局聚合：

服务器端的聚合方式与 FedAvg 保持完全一致：

$$w^{t+1} = \sum_{k=1}^{K} p_k w_k^{t+1} = w^t + \sum_{k=1}^{K} p_k \Delta_k$$

4. FedNova (Federated Normalized Averaging)

由于客户端算力和网络状态的异构性，各客户端单轮执行的本地步数 τ_k 往往是不相等的。直接使用 FedAvg 对权重求平均会隐式地给予步数多的客户端更高的权重，导致收敛到有偏的全局解（Objective Inconsistency）。

推导过程：

1. **定义归一化梯度：**

对于步数为 τ_k 的客户端 k，其本地总更新量为 $\Delta_k = w^t - w_k^{t+1}$。为了消除步数带来的尺度差异，计算其平均每步的**归一化伪梯度** d_k：

$$d_k = \frac{\Delta_k}{\tau_k} = \frac{w^t - w_k^{t+1}}{\tau_k}$$

2. **计算全局有效步数：**

系统层面的平均迭代步数 τ_{eff} 等于各个客户端步数的加权期望：

$$\tau_{eff} = \sum_{k=1}^{K} p_k \tau_k$$

3. **消除步数偏置的全局聚合：**

服务器使用全局有效步数 τ_{eff}，沿着归一化梯度 d_k 的加权平均方向更新全局模型：

$$w^{t+1} = w^t - \tau_{eff} \sum_{k=1}^{K} p_k d_k$$

将 d_k 代入并展开：

$$w^{t+1} = w^t - \left(\sum_{k=1}^{K} p_k \tau_k \right) \sum_{k=1}^{K} p_k \left(\frac{w^t - w_k^{t+1}}{\tau_k} \right)$$

4. **转换为增量相加的形式：**

$$w^{t+1} = w^t + \left(\sum_{k=1}^{K} p_k \tau_k \right) \sum_{k=1}^{K} p_k \frac{\Delta_k}{\tau_k}$$

推导结论： FedNova 通过将每个客户端的更新量除以其自身的步数 τ_k，再乘以全局平均步数 τ_{eff}，从数学上彻底解耦了“参与数据量比例”与“本地迭代步数”之间的耦合，保证了无论系统异构性多强，模型都严格按照全局损失函数的方向收敛。

5. 算法演进与对比分析

算法	本地优化目标	全局聚合公式（增量视角）
FedSGD	$$\min F_k(w)$$	$w^{t+1} = w^t - \eta \sum p_
FedAvg	$\min F_k(w)$	$w^{t+1} = w^t + \sum p_k \D
FedProx	$\min F_k(w) + \frac{\mu}{2} \Vert{}w - w^t\Vert{}^2$	$w^{t+1} = w^t + \sum p_k \D
FedNova	$\min F_k(w)$	$w^{t+1} = w^t + \tau_{eff} \s